Viva Question Bank

Smart City Utilities & Infrastructure Management System: prototype version

Dhanush & Gagan · DBMS Level 3 mini-project · 26 October 2026

How to use this
1. The project in one minute
2. Architecture and technology
3. ER diagram and schema design
4. Constraints (show them on the “Database rules” page)
5. SQL queries (show them on the “SQL lab” page)
6. Procedures, functions and triggers
7. Transactions, ACID and concurrency
8. MongoDB
9. Combining the two databases
10. Anomaly detection and the disaster simulator
11. Authentication, authorisation and security
12. Web app, testing and Git
13. Limitations and future work
14. Curveball questions
One-page cheat sheet

How to use this

Each answer starts with a **one-line answer** you can say first. If the examiner wants more, continue with the detail underneath. Every number in here matches what the prototype actually shows (seed 42), so you can open the app and point at it.

> **The most important rule for today:** be honest about what this version is. It is a working browser prototype. The two databases are simulated inside the page with the same tables, columns and rules as our design. The final build runs on real PostgreSQL, MongoDB and Next.js. Examiners respect a clear “this is the prototype, this is the plan” far more than an over-claim they can catch.

---

1. The project in one minute

Q1. What is your project?

One line: A system that runs a city’s electricity, water and gas: it reads smart meters, bills residents correctly, and catches leaks and outages automatically.

Three kinds of users log in. Residents see their usage and bills and can pay. Maintenance staff fix problems. City admins watch the whole city, run billing and test disasters.

Q2. What problem does it solve?

One line: Cities keep billing data and sensor data in separate systems, so leaks are found late and bills can’t be trusted or explained.

We connect the two: sensor data (MongoDB) feeds billing (PostgreSQL) and anomaly detection, and everyone sees one consistent picture.

Q3. Why is this a DBMS project and not just a website?

One line: The important logic lives in the database: keys, constraints, triggers, procedures, row-level security and aggregation pipelines.

Wrong bills, duplicate payments, illegal ticket changes and cross-resident data access are all stopped by the database itself, not only by the app.

Q4. What exactly is “this version”?

One line: A clickable prototype with synthetic data that demonstrates every workflow and every database rule we designed.

It is a single web page. Inside it, a JavaScript object called `PG` holds the 15 PostgreSQL tables and another called `MG` holds the 5 MongoDB collections. A small rule engine enforces the same CHECK, UNIQUE, EXCLUDE and trigger rules as our DDL and returns errors in PostgreSQL’s real message format. The side panel (“Under the hood”) shows the exact SQL or MongoDB statement each click would run in the real build.

Q5. So it is not connected to a real database?

One line: Correct. This version simulates both databases in memory; the real PostgreSQL 17 and MongoDB 8 build is the next phase of our 14-day plan.

Why we did it this way: it let us test the schema, the rules and the user flows quickly and show the complete end-to-end story before writing the backend. The table names, columns, constraints and queries in the prototype are the ones we will create for real.

Q6. What is the demo data?

One line: Fully synthetic: 7 zones, 6 residents, 8 properties, 24 meters, 4 billing cycles, 8 tariff slabs, 5 user accounts.

Names, emails (`@example.test`) and IDs (`SYN-00001`) are made up. No real government ID is stored anywhere. The data is generated with a fixed random seed (42), so every run starts the same.

2. Architecture and technology

Q7. Why two databases?

One line: Money needs strict rules and transactions (PostgreSQL); sensor readings are huge, fast and differently shaped per meter (MongoDB).

This is called **polyglot persistence**: using the right database for each workload.

	PostgreSQL	MongoDB

Stores | residents, properties, meters, tariffs, bills, payments, tickets, users | raw meter readings, anomaly events, ticket notes, notifications |
Strength | ACID transactions, joins, constraints, procedures, RLS | flexible documents, time-series storage, fast bulk inserts, aggregation pipelines |
If it fails | a citizen is overcharged | an alert is late |

Q8. Why PostgreSQL and not MySQL?

One line: PostgreSQL has Row-Level Security and exclusion constraints, which we use; MySQL has neither.

We also use `NUMERIC` for exact money, generated columns, partial unique indexes and recursive CTEs.

Q9. Why MongoDB?

One line: Electricity, water and gas meters send different fields, and readings arrive every few minutes forever; documents and time-series collections fit that.

An electricity reading has `kwh_total, voltage, current, power_factor`; a water reading has `volume_l_total, flow_lpm, pressure_psi`. In SQL that means many NULL columns or one table per meter type.

Q10. What is the “bridge” between the two databases?

One line: The `meter_reading` table: one row per meter per month, computed from MongoDB and stored in PostgreSQL for billing.

Millions of raw readings stay in MongoDB; PostgreSQL only gets the clean monthly total it needs.

Q11. What is the planned final tech stack?

One line: Next.js (TypeScript) for UI and API, PostgreSQL 17, MongoDB 8, Zod validation, Recharts, Vitest/Playwright tests and GitHub Actions CI.

No separate Express server: Next.js route handlers are the API, so auth and validation are written once. No Redis: we have not measured a need for a cache.

Q12. What is the prototype built with?

One line: Plain HTML, CSS and JavaScript in one file, with hand-drawn SVG charts and no libraries.

It was the fastest way to make something anyone can open in a browser. The final interface is Next.js as the rubric requires.

Q13. Walk me through the data flow.

1. Meters send a reading every 15 city-minutes → `db.meter_readings.insertMany(...)`.
2. Anomaly rules run on every batch → `anomaly_events` (MongoDB) → ticket (PostgreSQL).
3. Month end: aggregation pipeline computes each meter’s opening and closing register → `meter_reading` (PostgreSQL).
4. `sp_generate_invoice` applies tariff slabs → `invoice` + `invoice_line_item`.
5. Resident pays → `sp_record_payment` → trigger marks invoice PAID.
6. Dashboards read both databases and merge the results.

3. ER diagram and schema design

Q14. How many tables and which ones?

One line: 15 tables: the 10 from our original design plus 5 we added.

Original: `resident, property, utility_meter, billing_cycle, utility_rate, meter_reading, invoice, invoice_line_item, payment, maintenance_ticket`. Added: `zone, app_user, user_session, ticket_status_history, aggregation_run`.

Q15. Why did you add those 5?

- `zone`: avoids typos like “North” vs “north” and gives a City → District → Ward hierarchy (used by the recursive query).
- `app_user` and `user_session`: login, roles and sessions, needed for authentication and database-level access control.
- `ticket_status_history`: an audit trail of every ticket change, written by a trigger.
- `aggregation_run`: records each MongoDB → PostgreSQL transfer so it can be safely re-run.

Q16. Explain the main relationships.

One line: A resident owns many properties; a property has many meters, invoices and tickets; an invoice has many line items and payment attempts.

- Resident 1 : M Property
- Property 1 : M Utility_meter
- Utility_meter 1 : M Meter_reading (one per billing cycle)
- Property 1 : M Invoice; Billing_cycle 1 : M Invoice
- Invoice 1 : M Invoice_line_item; Utility_rate 1 : M Invoice_line_item
- Invoice 1 : M Payment, but at most one SUCCEEDED
- Zone 1 : M Zone (self-relationship: parent → children)
- Resident 1 : 0..1 App_user

Q17. What is a primary key, foreign key and candidate key here?

- **Primary key:** uniquely identifies a row, e.g. `invoice_id`.
- **Foreign key:** points to a primary key in another table, e.g. `invoice.property_id → property.property_id`.
- **Candidate/alternate key:** another column that is also unique, e.g. `resident.email`, `invoice.invoice_no`, `utility_meter.meter_code`.
- **Composite unique key:** `(property_id, cycle_id)` on invoice, `(meter_id, cycle_id)` on meter_reading.

Q18. Why UUIDs as primary keys?

One line: They can't be guessed in a URL and can be created anywhere without a central counter.

Trade-off: they are bigger than integers. Small lookup tables (`zone`, `billing_cycle`, `utility_rate`) use integers. Human-friendly codes (`P-0001`, `INV-2026-09-0017`, `MT-0004`) are separate UNIQUE columns.

Q19. Is your schema normalised? To which form?

One line: Third Normal Form: every non-key column depends on the key, the whole key, and nothing but the key.

Example: a resident's email is stored only in `resident`, never copied into `property` or `invoice`. That prevents update, insert and delete anomalies.

Q20. Explain 1NF, 2NF, 3NF with your tables.

- **1NF:** no repeating groups. Each line of a bill is its own row in `invoice_line_item`, not columns `elec1, elec2, water1...`.
- **2NF:** no partial dependency on part of a composite key. In `meter_reading`, the key is `(meter_id, cycle_id)`; the property is not stored there because it depends on `meter_id` alone.
- **3NF:** no transitive dependency. Zone name lives in `zone`, not in `property`, because it depends on `zone_id`, not on the property.

Q21. Did you denormalise anywhere? Why?

One line: Yes, two deliberate snapshots: `invoice.total_amount` and `invoice_line_item.rate_applied`.

A bill must keep the price that applied on the day it was issued, even if the tariff changes next month. A trigger keeps `total_amount` equal to the sum of its lines so it can never drift.

Q22. Why is `total_units_consumed` a generated column?

One line: It's always `closing_register − opening_register`, so storing it separately could only introduce mistakes.

`GENERATED ALWAYS AS (closing_register - opening_register) STORED`.

Q23. Why ON DELETE RESTRICT on most foreign keys?

One line: Deleting a resident must never silently erase their bills and payments.

We use RESTRICT on money and ownership paths, CASCADE only where a child can't exist alone (line items, ticket history, sessions), and SET NULL where the child should survive (a ticket keeps existing if its staff account is removed). Residents are soft-deleted with `is_active = false`.

Q24. What did you change from your original schema, and why?

Original	Problem	Our fix

Billing summed the meter's kWh value | That value is a running total (like a car's odometer), so summing gives absurd bills | Usage = closing − opening register |
No UNIQUE on invoice (property, cycle) | Same bill could be created twice | UNIQUE constraint + idempotent procedure |
Payment strictly 1:1 with invoice | Failed attempts and retries can't be stored | Payment stores every attempt; partial unique index allows one success |
`max_units NOT NULL` on tariffs | Top slab can't be "and above" | NULL means no upper limit |
`government_id` | Real personal data | Synthetic `citizen_ref` with `CHECK (LIKE 'SYN-%')` |
Free-text `zone_type` | Typos split analytics | `zone` table with hierarchy |
Tickets in both SQL and Mongo | Two sources of truth | Ticket in SQL, notes in Mongo |

Q25. Is this the most optimised schema?

One line: It's not the smallest; it's the one best fitted to our workload: correct money, fast common queries, and no anomalies.

Indexes follow the real queries, rules live in the database, and raw sensor data stays out of PostgreSQL so its tables stay small. At real scale we would partition invoices by billing cycle.

4. Constraints (show them on the "Database rules" page)

Q26. What constraints did you use?

NOT NULL, DEFAULT, CHECK, UNIQUE, PRIMARY KEY, FOREIGN KEY, partial UNIQUE index and EXCLUDE.

Q27. Show me a CHECK constraint working.

Database rules → "Billing cycle with month 13" → `ERROR: new row for relation "billing_cycle" violates check constraint "billing_cycle_month_check"`. Other checks: `price_per_unit > 0`, `cycle_start_date < cycle_end_date < due_date`, `closing_register >= opening_register`, `citizen_ref LIKE 'SYN-%'`, ticket `status IN (...)`.

Q28. How do you prevent duplicate bills?

One line: `UNIQUE (property_id, cycle_id)` on invoice, plus the procedure checks first and returns the existing bill.

Demo: Billing run → Generate invoices twice → “0 created, 8 already existed”. Database rules → “Second invoice for the same property and month” → `duplicate key value violates unique constraint "invoice_property_id_cycle_id_key"`.

Q29. What is a partial unique index? Where do you use one?

One line: A UNIQUE rule that only applies to rows matching a condition.

1. `payment (invoice_id) WHERE status = 'SUCCEEDED'`: many failed attempts are fine, but only one success per invoice.
2. `maintenance_ticket (anomaly_key) WHERE status IN ('OPEN','ASSIGNED','IN_PROGRESS')`: only one live ticket per problem, but history can keep many resolved ones.

Q30. What is an EXCLUDE constraint?

One line: It stops two rows from “overlapping”; we use it so two versions of the same tariff slab can’t be valid on the same day.

`EXCLUDE USING gist (utility_type WITH =, tier_level WITH =, daterange(effective_start_date, effective_end_date) WITH &&)`. `&&` means “overlaps”. It needs the `btree_gist` extension. Demo: “Overlapping electricity tariff” → `conflicting key value violates exclusion constraint`.

Q31. Why put rules in the database instead of the code?

One line: The database is the last line of defence: every app, script or person that writes data has to obey it.

Code can have bugs or be bypassed; a constraint can’t.

5. SQL queries (show them on the “SQL lab” page)

Q32. What DML operations do you use?

- **INSERT:** new complaint, payment attempt, meter readings from the bridge.
- **UPDATE:** ticket status, payment status, invoice marked PAID.
- **DELETE:** logout deletes the session row; expired sessions.
- **UPSERT:** `INSERT ... ON CONFLICT (meter_id, cycle_id) DO UPDATE` in the bridge.
- **UPDATE … RETURNING:** ticket transitions return the new status.

Q33. Explain the recursive query.

One line: It starts at one zone and keeps walking down to its children until there are none, then totals usage for all of them.

```
WITH RECURSIVE subtree AS (
  SELECT zone_id FROM zone WHERE zone_id = $1            -- anchor: the chosen zone
  UNION ALL
  SELECT z.zone_id FROM zone z
```

```
          JOIN subtree s ON z.parent_zone_id = s.zone_id         -- recursive step
        )
        SELECT ... FROM subtree JOIN property ... GROUP BY utility_type;
```

Two parts: the **anchor** (starting row) and the **recursive member** (joins back to the CTE). It stops when the recursive step returns no new rows. North District → Ward N-1, Ward N-2.

Q34. What is a correlated subquery? Show yours.

One line: A subquery that uses a value from the outer row, so it runs once per outer row.

"Properties using more than 1.3× the average of other properties in the same district": the inner `SELECT AVG(...)` uses `p.zone_id` and `p.property_id` from the outer query. Result on our data: **P-0004 (Desai Bakery), 1,240 kWh vs 475.67 average**.

Q35. Difference between nested and correlated subqueries?

A **nested** subquery runs once and its result is reused (`WHERE resident_id IN (SELECT resident_id FROM resident WHERE is_active)`). A **correlated** one depends on the outer row and runs per row.

Q36. Explain your JOIN query for overdue residents.

One line: It joins resident → property → zone → invoice, keeps OVERDUE bills, groups by resident and zone, and keeps those with at least N overdue bills.

`GROUP BY ... HAVING COUNT(*) >= $1`. With N = 2 the answer is **Rahul Desai, Ward S-1, 2 bills, ₹19,271.50**. The source design used 3 months; we made it a parameter for the small dataset.

Q37. WHERE vs HAVING?

WHERE filters rows before grouping; HAVING filters groups after `GROUP BY` (so it can use `COUNT`, `SUM`).

Q38. Types of JOIN you use?

INNER JOIN for normal relationships; LEFT JOIN where a side may have no rows (zones with no unpaid bills should still show ₹0); self-join on `zone` through the recursive CTE.

Q39. What is a CTE? Why use one?

One line: A named temporary result (`WITH name AS (...)`) used inside one query; it makes complex queries readable step by step.

Our ageing report: `open_bills` CTE calculates days late, then the main query buckets them into "not yet due / 1–30 / 31–60 / 60+ days".

Q40. Explain the window function query.

One line: `LAG(kwh) OVER (PARTITION BY property ORDER BY cycle)` gives each row the previous month's value, so we can compute month-over-month change without a self-join.

Top result: **P-0004, August 724 → September 1,240 kWh, +71.3%** (we seeded an "oven upgrade" there).

---

6. Procedures, functions and triggers

Q41. Function vs procedure?

A **function** returns a value and can be used inside a query (`SELECT fn_tiered_charge(...)`). A **procedure** is called with `CALL`, can manage transactions, and doesn't need to return anything (`CALL sp_generate_invoice(...)`).

Q42. How does tiered billing work? Do the maths.

One line: Units are split across slabs and each slab is charged at its own rate.

Electricity slabs: 0–100 @ ₹5, 100–300 @ ₹8, 300+ @ ₹12. Asha used 450 kWh in September: 100 × 5 = 500; 200 × 8 = 1,600; 150 × 12 = 1,800 → **₹3,900**. Her full September bill **₹4,860** = electricity 3,900 + water (18 kL: 10 × 8 + 8 × 15 = 200) + gas (24 m³: 20 × 30 + 4 × 40 = 760). For each slab the function takes `LEAST(units, max) − min`, and ignores negative results.

Q43. What does `sp_generate_invoice` do?

1. Takes an advisory lock for (property, cycle) so two runs can't race.
2. If the invoice already exists, returns it (no duplicate).
3. Checks every meter has a `meter_reading` for the cycle; if not, raises `MISSING_READING` and the whole transaction rolls back.
4. Calls `fn_tiered_charge` per utility and inserts one line per slab.
5. A trigger updates the invoice total. Demo: generate before aggregating → rollback error; aggregate, then generate → 8 invoices; generate again → 0 new.

Q44. What does `sp_record_payment` do?

- Same idempotency key seen before → returns the original payment, writes nothing.
- Invoice already PAID → `ALREADY_PAID`.
- Gateway declines → attempt saved as FAILED, invoice unchanged (savepoint + rollback).
- Gateway approves → SUCCEEDED; trigger marks invoice PAID. Payments are **simulated**, never real money; the UI says so.

Q45. List your triggers and why each exists.

Trigger	Fires on	Job
`trg_ticket_transition`	before ticket status update	Only allows OPEN → ASSIGNED → IN_PROGRESS → RESOLVED → CLOSED (and reopen); requires assignee and resolution notes
`trg_ticket_history`	after ticket insert/status update	Writes a row to `ticket_status_history` with who and when
`trg_invoice_total`	after line item change	Keeps invoice total = sum of lines; blocks edits on PAID invoices
`trg_payment_settles_invoice`	after payment becomes SUCCEEDED	Sets invoice PAID and `paid_at`
`trg_reading_locked`	before meter_reading update/delete	Blocks changes once that month is billed
`trg_tariff_contiguity`	deferred, on tariff change	Slabs must start at 0, have no gaps, only top slab unbounded

Q46. Show a trigger rejecting something.

As Ravi, press “Try to close directly” on an OPEN ticket → `ILLEGAL_TRANSITION: OPEN -> CLOSED is not allowed`. Or Database rules → “Change a meter reading that’s already billed” → `READING_LOCKED`.

Q47. BEFORE vs AFTER trigger?

BEFORE runs before the row is written, so it can change or reject it (our transition check). AFTER runs once the row is written, so it reacts to it (history row, invoice total, marking PAID).

Q48. Row-level vs statement-level trigger?

Ours are `FOR EACH ROW`: they run once per changed row because each ticket or payment needs its own check.

7. Transactions, ACID and concurrency

Q49. Explain ACID with your project.

- **Atomicity:** invoice + all its line items are created together or not at all.
- **Consistency:** constraints and triggers keep totals, statuses and keys valid.
- **Isolation:** two admins generating the same bill don’t interfere (advisory lock + UNIQUE).
- **Durability:** once a payment commits, it survives a crash.

Q50. Where do you use ROLLBACK?

In `sp_generate_invoice` when any meter reading is missing, and in `sp_record_payment` when the gateway declines (`ROLLBACK TO SAVEPOINT`), so money is never marked paid without a successful payment.

Q51. What is idempotency? Why does it matter?

One line: Doing the same request twice has the same effect as doing it once.

A resident double-clicking “Pay” must not be charged twice. Each payment request has an `idempotency_key` (UNIQUE). Demo: “Send the same request twice” → one payment row.

Q52. What if two people generate invoices at the same time?

The advisory lock makes the second wait; when it continues, the invoice already exists so it’s returned. Even without the lock, `UNIQUE (property_id, cycle_id)` would reject the second insert.

8. MongoDB

Q53. What collections do you have?

Collection	Holds	Operations shown
`meter_readings`	every raw reading (time-series)	insertMany, find by meter and time, aggregate
`anomaly_events`	detected problems with reasons	upsert, acknowledge (`$set`)

`maintenance_logs` | ticket notes and sensor snapshots | `$push` note, `$pull` delete note |
`notifications` | resident alerts | insert, mark read (`$set`), dismiss (`deleteOne`) |
`ingestion_batches` | simulator batch records | insert, skip duplicates |

Q54. Show CRUD / RUD on MongoDB.

- **Create:** readings `insertMany`, ticket log `insertOne`.
- **Read:** live chart `find({ "meta.meter_code": ..., ts: { $gte: ... } }).sort({ ts: 1 })`; aggregations on the admin page.
- **Update:** Acknowledge an anomaly (`$set`), add a field note (`$push`), mark alert read (`$set`).
- **Delete:** dismiss an alert (`deleteOne`), delete a note (`$pull`), old readings expire via TTL.

Q55. What is a time-series collection?

One line: A MongoDB collection optimised for measurements over time; it groups readings into compressed buckets by meter and time.

Created with `{ timeField: "ts", metaField: "meta", granularity: "minutes" }`. `meta` holds meter code, property, utility and zone.

Q56. Show me a document.

```
        { "ts": "2026-10-25T09:15:00Z",
          "meta": { "meter_code": "MTR-W-0001", "property_code": "P-0001", "utility": "WATER", "zone": "Ward
N-1" },
          "volume_l_total": 712480, "flow_lpm": 14.6, "pressure_psi": 24.3 }
```

An electricity document has `kwh_total, voltage, current, power_factor` instead. Open “Inside the databases” → MongoDB → `meter_readings` to show live ones.

Q57. Which indexes and why?

`{ "meta.meter_code": 1, ts: -1 }` for “latest readings of this meter” (resident chart, meter health); `{ "meta.zone": 1, ts: -1 }` for zone totals; `{ status: 1, detected_at: -1 }` on anomaly events for the alert feed. Every index matches a query we actually run.

Q58. Explain an aggregation pipeline you use.

One line: For billing, it sorts each meter’s readings by time and takes the first and last register value in the month.

```
        db.meter_readings.aggregate([
          { $match: { ts: { $gte: ISODate("2026-10-01"), $lte: now } } },
          { $sort:  { "meta.meter_code": 1, ts: 1 } },
          { $group: { _id: "$meta.meter_code",
              opening: { $first: "$kwh_total" }, closing: { $last: "$kwh_total" }, docs: { $sum: 1 } } } ])
```

`$match` filters (like WHERE), `$sort` orders, `$group` groups (like GROUP BY). Map-Reduce, from the original plan, is deprecated, so we use pipelines.

Q59. Why first/last register instead of summing readings?

One line: The meter reports a running total; usage is just “end minus start”.

A duplicated or late reading has a value already inside that range, so it can’t change the bill. A sum would double-count it.

Q60. Is MongoDB schemaless? How do you validate?

One line: Flexible, not lawless: `$jsonSchema` validators reject bad documents, and the API validates with Zod first.

Demo: Database rules → “Malformed meter reading” → `Document failed validation` (missing `meta.meter_code`, negative flow).

Q61. How do you handle old data?

Raw readings expire after 180 days with a TTL index; a daily roll-up (`readings_daily`) keeps the summary. Billing never needs old raw data because the monthly total is already in PostgreSQL. (The prototype keeps only the last 30 hours in memory.)

Q62. SQL vs NoSQL: when would you not use MongoDB?

For anything with money or strict relationships. Bills need multi-table transactions, foreign keys and exact decimals, which is why invoices live in PostgreSQL.

9. Combining the two databases

Q63. How do you keep PostgreSQL and MongoDB consistent?

One line: They can’t share one transaction, so every hand-off is designed to be safe to repeat.

- Bridge: `UNIQUE (meter_id, cycle_id)` + `ON CONFLICT DO UPDATE`, so re-running just refreshes rows; locked once billed.
- Anomaly → ticket: the MongoDB event has `synced_to_sql: false` until the ticket exists (an **outbox** flag); a retry picks up anything left false.
- The SQL ticket function is itself idempotent (partial unique index).

Q64. What happens if the system crashes between writing to MongoDB and PostgreSQL?

The anomaly event stays `synced_to_sql: false`. The next scan retries and either creates the ticket or finds the existing one. Nothing is lost and nothing is duplicated.

Q65. Which database wins if they disagree?

PostgreSQL for anything about money (it’s the system of record). MongoDB is the source for raw measurements. A reconciliation report compares the pipeline totals with `meter_reading`.

Q66. Where do both databases appear on one screen?

Asha’s dashboard: property and unpaid bills from PostgreSQL, live chart and “October so far” from MongoDB. Admin “Wards” table: live demand from MongoDB, tickets and unpaid bills from PostgreSQL. Each panel is labelled with its source.

10. Anomaly detection and the disaster simulator

Q67. How do you detect a leak?

One line: If water keeps flowing above 2 L/min for 4 readings in a row (60 minutes), it's flagged as a CRITICAL leak.

A normal home uses about 0.1–0.7 L/min and drops near zero at night; a burst pipe flows about 14 L/min non-stop and pressure falls.

Q68. And spikes and outages?

- **Spike:** electricity above 3× its own 24-hour average and above 3 kW → HIGH.
- **Missing data:** a property's meters send nothing for 45+ minutes → MEDIUM, one ticket per property.

Q69. Is this AI or machine learning?

One line: No: simple statistical rules on purpose, because every alert must be explainable.

Every ticket stores the reason in words, e.g. "Water kept flowing above 2 L/min for 60+ min (now 14.6 L/min, pressure 24.3 psi)". Machine learning is a possible extension once the baseline rules work.

Q70. How do you avoid 100 tickets for one leak?

The partial unique index on `anomaly_key` (e.g. `LEAK:MTR-W-0001`) allows only one active ticket. Repeats increase `occurrence_count` and `last_seen_at` instead. On the demo, the leak ticket shows "seen 24×" but there is one ticket.

Q71. What happens after staff resolve it?

Ticket → RESOLVED (resolution notes required), history row written, the MongoDB event is marked RESOLVED, the property goes back to NORMAL, and the simulated meter returns to normal flow. If it leaks again, a new ticket is allowed.

Q72. How does the simulator work?

Each meter has a daily usage pattern (morning and evening peaks for homes, business hours for shops) plus small random noise from a seeded generator. Every 2 real seconds = 15 city minutes. Scenarios override a meter's behaviour (burst, spike) or stop meters reporting (outage). "+1 hour" runs 4 ticks instantly.

11. Authentication, authorisation and security

Q73. How many roles?

Three: Resident, Maintenance staff and Municipal admin (the rubric needs two). Demo accounts: Asha and Vikram (residents), Ravi (staff, North) and Kavya (staff, South), and the admin.

Q74. How does login work?

One line: The password is checked against an Argon2id hash; a random session token goes in an HttpOnly cookie and only its SHA-256 hash is stored in `user_session`.

Prototype note: every demo account uses `demo123` and the hash is not really computed; the statements in the side panel show what the real build runs.

Q75. Why hash passwords? Why Argon2id?

So a database leak doesn't reveal passwords. Argon2id is deliberately slow and memory-hungry, which makes guessing very expensive. (bcrypt is the acceptable alternative.)

Q76. What is "role-based access control in the database"?

One line: PostgreSQL itself decides what each role can see and do, using roles, GRANTs and Row-Level Security.

Roles: `resident_role`, `staff_role`, `admin_role`. Each request runs `SET LOCAL ROLE resident_role` and `set_config('app.user_id', ...)`. GRANTs control actions (a resident can SELECT invoices but not UPDATE them). RLS policies control rows.

Q77. What is Row-Level Security? Show it.

One line: A policy that filters rows per user, inside the database.

```
CREATE POLICY invoice_resident_read ON invoice FOR SELECT TO resident_role
USING (property_id IN (SELECT p.property_id FROM property p
       JOIN app_user u ON u.resident_id = p.resident_id
       WHERE u.user_id = current_setting('app.user_id')::uuid));
```

Demo: as Asha, "Try to open Vikram's invoice" → 0 rows. As Ravi vs Kavya: each staff member sees only their district's tickets. "Inside the databases" shows fewer rows for non-admin roles.

Q78. Difference between frontend, backend and database protection?

Layer	Example	Can be bypassed?

Frontend | Residents don't see the admin menu | Yes, it's only convenience |
Backend | API checks the session and role | Only by a bug in our code |
Database | RLS returns 0 rows for other people's data | No |

Q79. How do you stop SQL injection?

Parameterised queries only (`WHERE email = $1`), never string concatenation. The SQL lab runs a fixed list of queries, never user-typed SQL.

Q80. And MongoDB injection?

Filters are built from validated plain values, so someone can't send `{ "$gt": "" }` as a password or ID. Keys starting with `$` are rejected.

Q81. Why return 404 instead of 403 for someone else's invoice?

So an attacker can't even learn that the invoice exists.

Q82. Other security measures?

Secrets in `.env` (never in Git); separate least-privilege database users (the meter writer can only insert readings); HttpOnly, Secure, SameSite cookies; account lockout after 5 failed logins; synthetic data only; payments clearly simulated.

12. Web app, testing and Git

Q83. Why Next.js for the final build?

The rubric requires it, and it gives us pages and API in one TypeScript project. Route handlers act as the backend, so no separate Express server is needed.

Q84. How will you test it?

Unit tests for billing maths and anomaly rules; database tests that each constraint rejects bad data; RLS tests (resident A gets 0 rows for B); API tests; four end-to-end browser tests (resident isolation, payment, staff resolves a ticket, pipe burst). GitHub Actions runs lint, type-check, tests and build on every pull request.

Q85. How did you test the prototype?

We clicked through every flow in a real browser: sign in, pay twice, failed payment, row-level security, pipe burst to ticket to resolve, aggregation, invoice generation twice, every database-rule demo, every SQL lab query, and the phone layout.

Q86. How do you split the work?

Dhanush leads PostgreSQL (schema, procedures, triggers, RLS, auth). Gagan leads MongoDB, the simulator and the interface. Each reviews the other’s work daily so both can explain and modify everything.

13. Limitations and future work

Q87. What are the limitations of this version?

- Databases are simulated in the browser; data resets when the page reloads.
- Login isn’t real authentication (shared demo password, no real hashing).
- Payments are simulated; no real gateway.
- Only 8 properties and 24 meters; we make no performance claims.
- Interface is plain HTML/JS, not yet Next.js.

Q88. What’s next?

Create the real PostgreSQL schema and MongoDB collections, build the Next.js app and API, move the rules from the JavaScript engine into actual constraints, triggers, procedures and RLS policies, then add automated tests and CI. That is our 14-day plan.

Q89. How would this scale to a real city?

Partition invoices and readings by month; shard MongoDB by meter; add read replicas for dashboards; run detection as a background worker with a queue; add a cache only if measurements show it's needed.

Q90. What would you add with more time?

A map view of zones, email/SMS alerts, PDF bills, machine-learning anomaly scores alongside the rules, late-fee automation and weather-driven demand forecasts.

14. Curveball questions

Q91. "Change the electricity tier 1 price live."

In the final build: `UPDATE utility_rate SET effective_end_date = '2026-11-01' WHERE ...;` then `INSERT` a new row from that date. We never edit an old price because issued bills must stay unchanged; the EXCLUDE constraint stops overlaps. Say this even if you can't run it in the prototype.

Q92. "Add a column, e.g. a second phone number."

```
ALTER TABLE resident ADD COLUMN alt_phone VARCHAR(15) CHECK (alt_phone ~ '^\+?[0-9]{10,14}$');
```

Q93. "Why not store everything in PostgreSQL with JSONB?"

Possible at small scale, and a fair alternative. We chose MongoDB because the rubric asks for a NoSQL database and because time-series collections, TTL and pipelines fit meter data. Money stays in PostgreSQL either way.

Q94. "Why not store everything in MongoDB?"

MongoDB does have transactions, but we'd lose foreign keys, CHECK/EXCLUDE constraints, exact NUMERIC money, joins and Row-Level Security, which are what keep bills correct and private.

Q95. "What happens if a meter is replaced and the register goes back to zero?"

Closing would be lower than opening, which the CHECK rejects. The bridge flags it as a meter malfunction, and staff record the old meter's final reading and the new meter's start (planned; not in the prototype).

Q96. "Your Q1 has joins across meters and invoices: doesn't it double-count?"

In the full design we total readings and invoices in separate CTEs before joining, exactly to avoid that fan-out. The prototype's Q1 totals only readings, so there is no double count.

Q97. "Is your data real?"

No. All names, emails, IDs, addresses and readings are synthetic and labelled as such.

Q98. “What did you learn?”

No single database fits every problem. The hard part isn’t using two databases, it’s making their hand-offs safe: unique keys, idempotent operations and an outbox so nothing is lost or doubled.

One-page cheat sheet

Topic	Say this
Project	City electricity, water and gas: meters → MongoDB, bills → PostgreSQL, leaks → tickets
This version	Browser prototype; both databases simulated with our real schema and rules
Tables	15 (10 original + zone, app_user, user_session, ticket_status_history, aggregation_run)
Collections	meter_readings (time-series), anomaly_events, maintenance_logs, notifications, ingestion_batches
Normal form	3NF; snapshots of price and total on purpose
Bridge	meter_reading: one row per meter per month = closing − opening register
Billing example	450 kWh → 100×5 + 200×8 + 150×12 = ₹3,900
No duplicates	UNIQUE (property_id, cycle_id); idempotency_key; partial unique index for one success, one live ticket
Recursive	Zone tree City → District → Ward
Correlated	P-0004 uses 1,240 kWh vs 475.67 district average
Window	LAG(): P-0004 +71.3% August → September
Triggers	ticket transitions, ticket history, invoice total, payment settles invoice, reading lock, tariff contiguity
Procedures	sp_generate_invoice, sp_record_payment; functions fn_tiered_charge, fn_open_anomaly_ticket
Security	3 roles; RLS + GRANTs in PostgreSQL; Argon2id; HttpOnly cookie; parameterised queries
Leak rule	> 2 L/min for 60 min → CRITICAL; one ticket per problem
Consistency	Idempotent hand-offs + outbox flag `synced_to_sql`